

Technical Specification and Prototyping Blueprint: AI Trip Planner

1. Executive Project Snapshot and Strategic Vision

The AI Trip Planner is engineered as a hybrid intelligence system, a strategic architecture designed to overcome the inherent reliability limitations of Large Language Models (LLMs) in temporal reasoning. By decoupling Natural Language Understanding (NLU) from itinerary scheduling, we provide a robust, low-latency mobile experience that is achievable within a strict one-month development cycle. This system leverages the linguistic power of Gemini for intent extraction while relying on deterministic Dart logic and local SQLite data for execution, ensuring that the final output is both mathematically sound and responsive.

Project Core Parameters

Component, Technology, Functional Role

Frontend Framework, Flutter / Dart, Cross-platform UI/UX utilizing Provider for state management.

Intelligence Layer, Gemini API, NLU to convert unstructured prompts into structured JSON parameters.

Local Persistence, SQLite (sqflite), "High-speed local storage for itineraries, activities, and pre-baked data."

Backend / Security, Firebase Auth, Secure user authentication and persistent session management.

External Data, Amadeus API, "Fetching real-world flight schedules, hotel lists, and routes."

Value Proposition Assessment: The Hybrid Model

The core engineering justification for this hybrid approach—rather than a pure AI-generated itinerary—rests on technical reliability:

- **Deterministic Scheduling:** LLMs frequently "hallucinate" time durations and logistical overlaps. Our custom Dart algorithm ensures 100% accuracy in time-blocking and sequencing.
- **Performance & Latency:** By querying a local SQLite database for trip synthesis, we minimize costly network round-trips, delivering an instantaneous user experience post-parsing.
- **Economic Scalability:** Using pre-baked trips and local data searches significantly reduces LLM token consumption and API costs during high-traffic scenarios.
- **Conversational Zero-Friction:** NLU parsing replaces complex, multi-page forms with a single input, drastically increasing user conversion and retention. This architectural vision establishes a foundation where intelligence serves as the entry point, while structured data logic serves as the backbone.

2. Functional Requirements and Intelligent Logic

In the current mobile landscape, NLU is a strategic differentiator. By eliminating the friction of manual form-filling, we reduce user drop-off. The system transitions from a "form-centric" model to an "intent-centric" model, allowing users to express complex needs in natural language.

The Intelligence Engine (LLM Integration)

The transformation of unstructured text into a functional itinerary follows a rigorous five-step process:

1. **Input Capture:** The user enters a conversational prompt (e.g., "I want to go to Paris by train for 2 days and see the Eiffel Tower").
2. **LLM Parameter Extraction:** The Gemini API parses the input into structured JSON, identifying key variables: Destination, Duration, Transport Mode, and Key Spots.
3. **Local Database Synthesis:** The app uses these parameters to query the local SQLite database and Amadeus API to retrieve specific transport schedules, hotel options, and point-of-interest (POI) details.
4. **Chronological Sequencing:** A custom Dart algorithm takes the retrieved data and organizes it into realistic time blocks (Transport → Lunch → Hotel Check-in → Sightseeing).
5. **Itinerary Interaction:** The final structured schedule is rendered as a scrollable, day-wise timeline for the user to explore, edit, and track.

Feature Impact Analysis

Core Feature, "Strategic Impact (The ""So What?"" Layer)"

NLU Parsing, Removes the barrier of entry; users can plan complex trips in seconds rather than minutes.

Time-Aware Generation, "Prevents logistical failures (e.g., overlapping flights and tours), ensuring the app provides professional-grade utility."

Smart Fallbacks, "Prevents app ""dead-ends"" by applying logical defaults when the LLM encounters unsupported cities or incomplete data."

Cost Estimation, Drives financial transparency by aggregating real-time price estimates across all activity categories.

Advanced Capabilities Critique

The inclusion of **Pre-baked Suggested Trips** is a critical architectural decision for this MVP. It serves as a zero-delay demo feature that bypasses LLM token costs and network latency entirely. By loading ready-made itineraries directly from local JSON storage, we ensure a "zero-fail" state for the 5th-semester project showcase. Furthermore, the **Start Trip Engine** transforms the app from a static planner into a dynamic travel companion by providing a reactive UI that tracks the user's progress in real-time. These intelligent features require a highly structured data layer to ensure persistence and cross-platform speed.

3. Data Architecture and Schema Design

A "local-first" data approach is mandatory for travel applications where intermittent connectivity is a standard user condition. We balance Firebase (for cloud-based identity) with SQLite (for high-performance local persistence).

SQLite Schema Specification

The relational structure maintains data integrity through a primary-foreign key relationship, ensuring that all activities are correctly anchored to a specific trip.

```
-- Table: trips
-- High-level metadata for each itinerary.
CREATE TABLE trips (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT,           -- e.g., "Paris Summer Trip"
  original_prompt TEXT, -- Raw user input for reference
  start_date TEXT,      -- ISO8601 format
  end_date TEXT,        -- ISO8601 format
  status TEXT DEFAULT 'UPCOMING', -- 'UPCOMING', 'ACTIVE', 'PAST'
  created_at TEXT       -- Generation timestamp
);

-- Table: activities
-- Individual events linked to a trip.
CREATE TABLE activities (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  trip_id INTEGER,      -- Foreign Key referencing trips.id
  day_number INTEGER,   -- e.g., Day 1, Day 2
  activity_type TEXT,   -- Enum: 'TRANSPORT', 'HOTEL',
'SIGHTSEEING', 'FOOD'
  title TEXT,           -- e.g., "Flight to Rome"
  description TEXT,     -- Details (e.g., "Train Name: TGV 9452")
  start_time TEXT,      -- ISO8601/Time format
  end_time TEXT,        -- ISO8601/Time format
  cost_estimate REAL,   -- Price tag for the card
  FOREIGN KEY (trip_id) REFERENCES trips (id) ON DELETE CASCADE
);
```

API Integration Strategy

To maintain a clean separation of concerns, external services are isolated into distinct logic layers:

- **ai_service.dart (The Brain):** Exclusively handles Gemini API communication and JSON parsing.

- **travel_service.dart (The Data Provider):** Manages Amadeus API calls for fetching real-world flights, hotels, and route data. This orchestration ensures that if an API provider changes, the core business logic and UI remain unaffected.

Proposed Folder Structure (MVVM)

To facilitate parallel development for a 3-person team, we will adhere to the following directory tree:

```
lib/
├── core/           # App-wide config, themes, constants
├── data/
│   ├── models/    # trip_model.dart, activity_model.dart
│   ├── services/  # ai_service.dart, travel_service.dart
│   └── local_db/   # db_helper.dart (SQLite management)
├── providers/      # State management (trip_provider.dart)
└── presentation/
    ├── screens/    # The 8 core screens
    └── widgets/    # timeline_card.dart, custom_text_fields
```

4. User Experience: The 8-Screen Prototype Architecture

The UI flow is designed to guide the user from unstructured intent to a managed execution state.

Screen-by-Screen Breakdown

- **Splash / Auth Gate:**
- **Goal:** Direct user based on session status.
- **Data Trigger:** Firebase authStateChanges() StreamBuilder.
- **Login & Sign-Up Screen:**
- **Goal:** Authenticate user.
- **Data Trigger:** Firebase Auth credential verification.
- **Home Screen:**
- **Goal:** Initiate AI generation or search destinations.
- **Data Trigger:** SELECT query to the trips table for saved itineraries.
- **Destination Details Screen:**
- **Goal:** Provide destination context (POI, tips).
- **Data Trigger:** Local SQLite lookup or Amadeus Search API via travel_service.dart.
- **Itinerary Timeline Screen:**
- **Goal:** Display the generated day-by-day plan.
- **Data Trigger:** Join query between trips and activities based on trip_id.
- **My Trips / Schedule Manager:**
- **Goal:** Manage trip lifecycle (Upcoming, Active, Past).
- **Data Trigger:** Filtered SQLite query on the status column.
- **User Profile Screen:**
- **Goal:** Manage account metadata.

- **Data Trigger:** FirebaseAuth.instance.currentUser metadata.
- **Spot / Location Details Sheet:**
- **Goal:** View granular activity details.
- **Data Trigger:** ID-based SQLite query to the activities table.

Timeline Visualization and Live Mode Logic

The timeline_card.dart is the UI's focal point, utilizing category icons (Bed, Plane, Camera, Plate) and time/cost tags for high scannability. The **"Start Trip" Engine** utilizes the device's system clock to drive the UI state. When a trip's status is set to "ACTIVE," the Itinerary Timeline Screen logic compares the current DateTime.now() with the start_time and end_time of each activity card in the activities table. If a match is found, the specific card receives a "pulsing indicator," providing an active, reactive planning experience.

5. Development Roadmap and Prototyping Phases

A phase-based approach ensures that technical debt is minimized and all three team members contribute to the full-stack architecture.

Phased Execution Plan

1. **Phase 1: Setup & Database:** Repository config, Firebase Auth connection, and SQLite db_helper.dart implementation.
2. **Phase 2: Auth & Core UI:** Implementation of the Auth Gate, Home Screen search bar, and Destination Detail layouts.
3. **Phase 3: LLM Integration & Logic:** Writing the ai_service.dart Gemini parser and the Dart scheduling algorithm.
4. **Phase 4: Timeline & Live Mode:** Styling the timeline_card.dart and implementing the real-time clock tracking logic.
5. **Phase 5: Polish & Documentation:** Handling edge cases (unsupported cities), pre-baked offline mode, and final presentation slides.

Critical Path Analysis

The most significant technical hurdles are the **Gemini API Parser** (ensuring consistent JSON schema) and the **Dart Scheduling Algorithm** (ensuring logical time-blocks).

Pair-programming is **mandated** for these modules to maintain logic consistency between the AI's "intent" and the app's "execution."

The Minimum Viable Product (MVP) Boundary

The MVP is defined as a functional 8-screen prototype utilizing the hybrid LLM/SQLite model. Features such as Google Maps Directions and real-time distance calculations are designated as "Phase 2" and are excluded from the current 1-month development scope to ensure project completion. **Summary Statement:** This specification serves as the definitive architectural contract for the development team. With the data schema finalized and the logic/LLM boundaries clearly defined, the project is officially cleared for Phase 1 execution.